Review Problem 15

- Orange runs at 1GHz, and provides a unit making all floating point operations take 1 cycle. Grape runs at 1.2 GHz by deleting the unit, meaning floating point operations take 20 cycles. Which machine is better?

Warning 1: Amdahl's Law

The impact of a performance improvement is limited by what is NOT improved:

$$\text{Execution time after improvement} = \text{Execution time of unaffected} + \text{Execution time affected} * \frac{1}{\text{Amount of improvement}}$$

Example: Assume a program runs in 100 seconds on a machine, with multiply responsible for 80 seconds of this time. How much do we have to speed up multiply to make the program run 4 times faster?

$$\frac{100}{4} = 20 + 80 \times \frac{1}{\text{improve}}$$

$$25 = 20 + \frac{80}{\text{improve}}$$

$$5 = \frac{80}{\text{Improve}} \quad \text{Improve} = \frac{80}{5} = 16\times$$

5 times faster?

$$\frac{100}{5} = 20 + \frac{80}{\text{improve}}$$

$$20 = 20 + \frac{80}{\text{Improve}}$$

$$0 = \frac{80}{\text{Improve}} \quad \text{Improve} = \infty$$

Warning 2: BIPs, GHz ≠ Performance

Higher MHz (clock rate) doesn't always mean better CPU

Orange computer: 1000 MHz, CPI: 2.5, 1 billion instruction program

$Exec = 1\times10^9 \times 2.5 \times \frac{1}{1\times10^9} = 2.5$ seconds

Grape computer: 500MHz, CPI: 1.1, 1 billion instruction program

$Exec = 1\times10^9 \times 1.1 \times \frac{1}{0.5\times10^9} = 2\times1.1 = 2.2$ seconds

Higher MIPs (million instructions per second) doesn't always mean better CPU

1 GHz machine, with two different compilers

Compiler A on program X: 10 Billion ALU, 1 Billion Load

Compiler B on program X: 5 Billion ALU, 1Billion Load

$Exec_A = (10\times10^9\times1 + 1\times10^9\times5)\cdot1ns = 15sec$

$Exec_B = (5\times10^9\times1 + 1\times10^9\times5)\cdot1ns = 10\ sec$

Execution Time: A 15 B 10

A: $\frac{11B\ inst}{15\ sec} = 0.73$ B: $\frac{6\ B\ inst}{10} = 0.6$

BIPS: A 0.73 B 0.6

Instruction Type	Type Cycles
ALU	1
Load	5
Store	3
Branch	2

Processor Performance Summary

Machine performance:

$$\text{CPU execution time for a program} = \text{Instructions for a program} * \text{CPI} * \frac{1}{\text{Clock rate}}$$

Better performance:

	Intel CISC	RISC
↓ number of instructions to implement computations	↓	↑
↓ CPI	↑	↓
↑ Clock rate	↓	↑

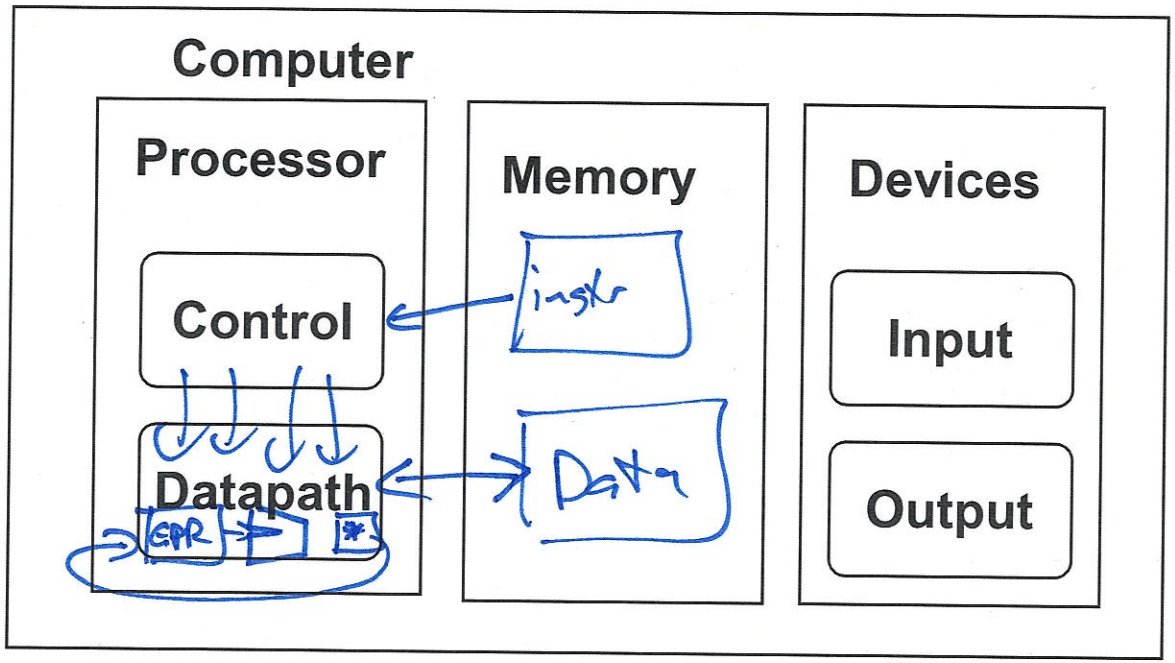
Improving performance must balance each constraint

Example: 1980's RISC vs. CISC

CISC: Complex Instruction Set Compute
RISC: Reduced Instruction Set Compute

Datapath & Control

Readings: 4.1-4.4



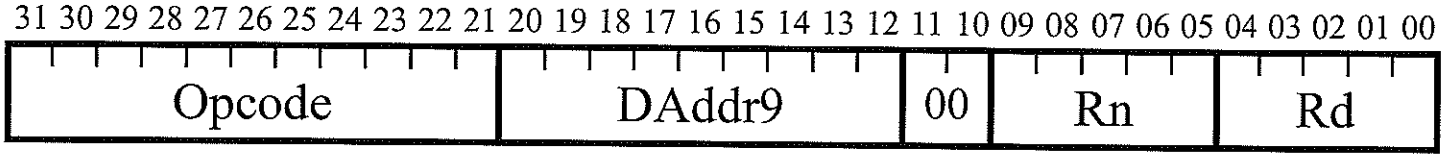
Datapath: System for performing operations on data, plus memory access.

Control: Control the datapath in response to instructions.

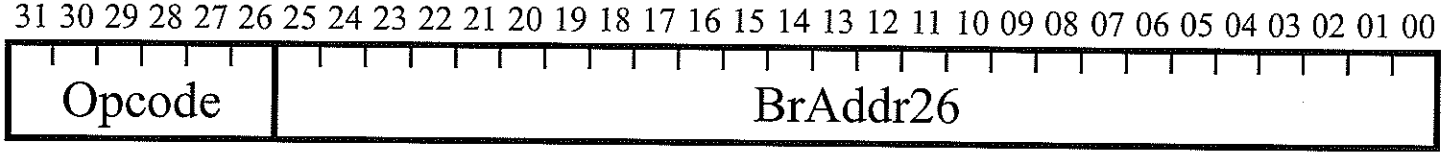
Simple CPU

Develop complete CPU for subset of instruction set

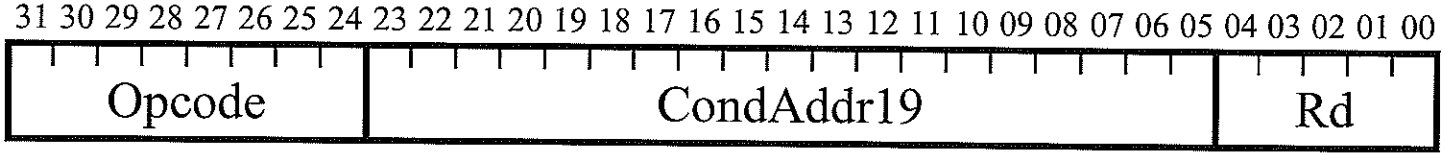
Memory: `LDUR, STUR`



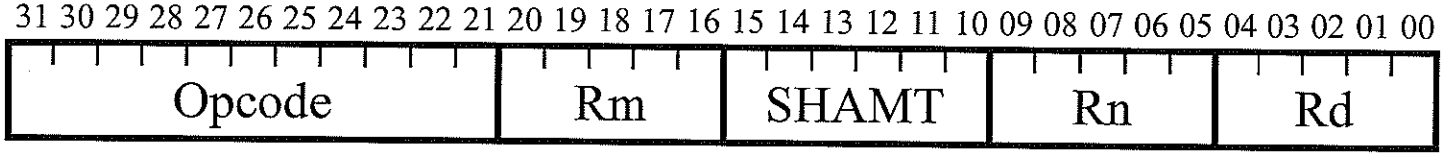
Branch: `B`



Conditional Branch: `CBZ`

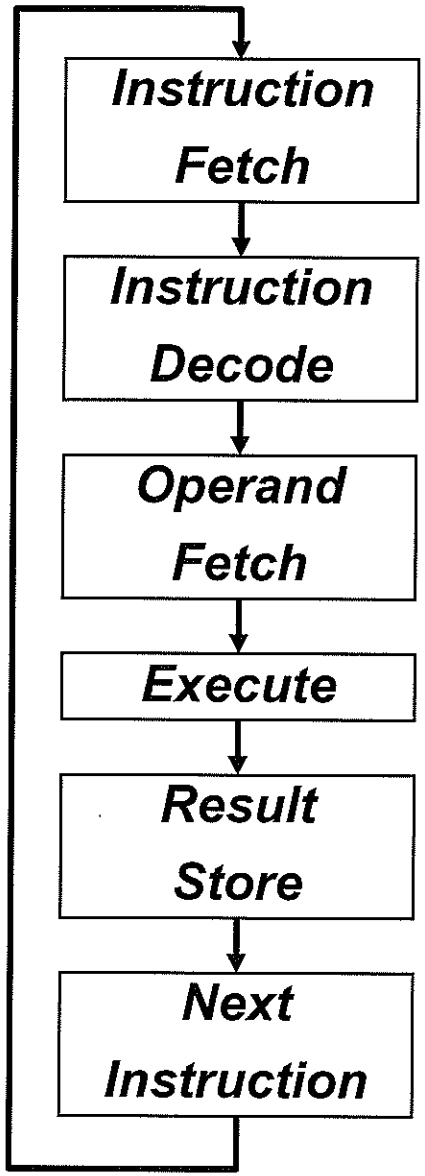


Arithmetic: `ADD, SUB`



Most other instructions similar

Execution Cycle



Obtain instruction from program storage

Determine required actions and instruction size

Locate and obtain operand data

Compute result value or status

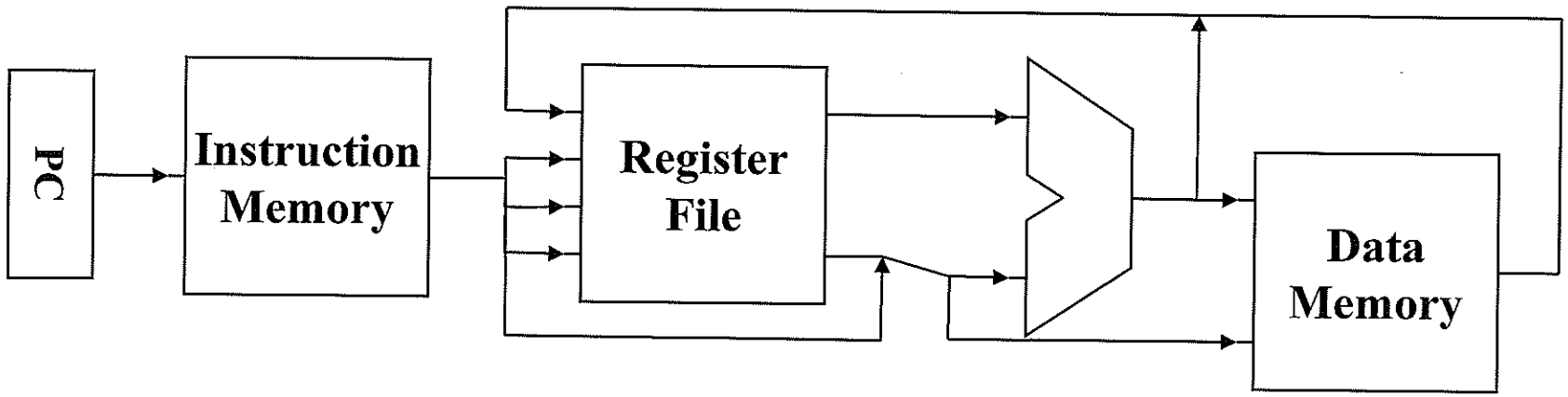
Deposit results in storage for later use

Determine successor instruction

Processor Overview

Overall Dataflow

- PC fetches instructions
- Instructions select operand registers, ALU immediate values
- ALU computes values
- Load/Store addresses computed in ALU
- Result goes to register file or Data memory



RTL & Processor Design

Convert instructions to Register Transfer Level (RTL) specification

```
RegA = RegB + RegC;
```

RTL specifies required interconnection of units, control

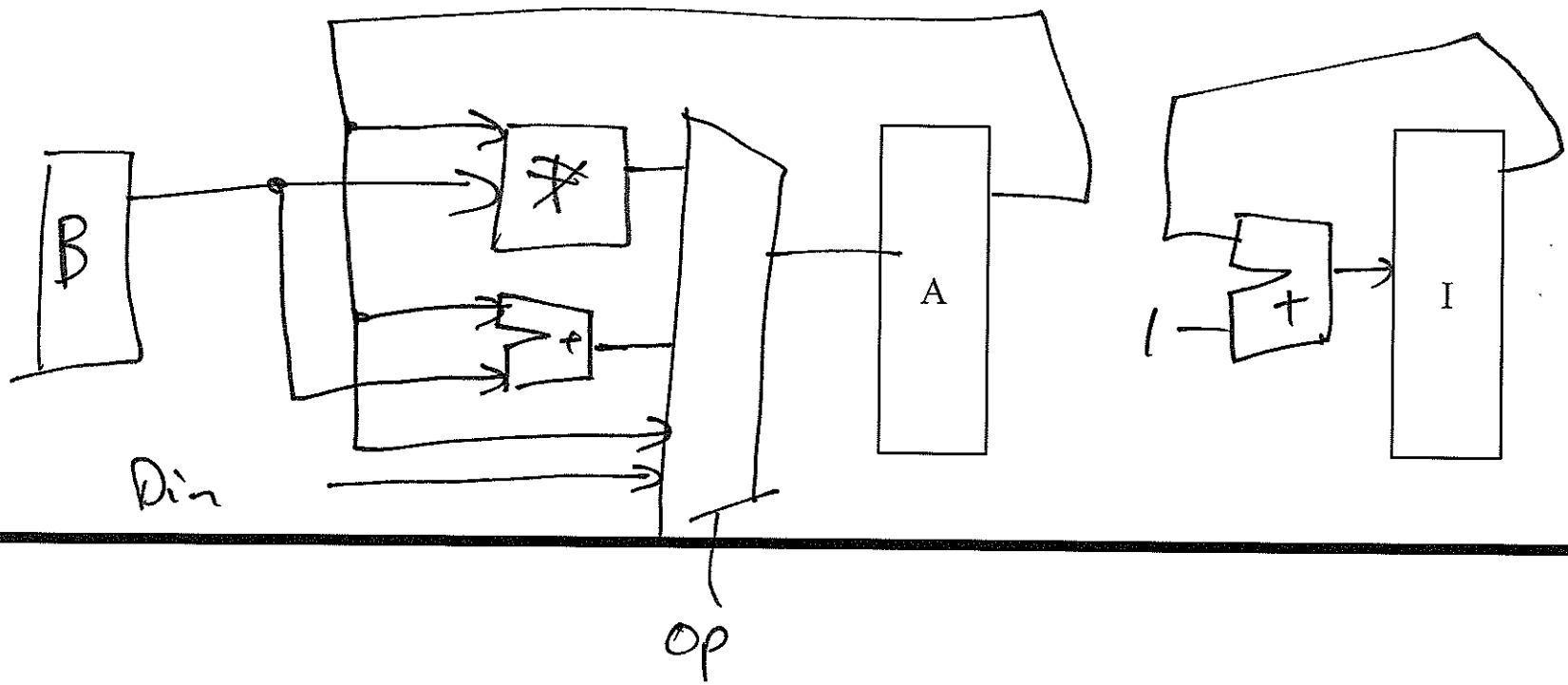
Math unit example:

(add): A = A + B; I++;

(mult): A = A * B: I++;

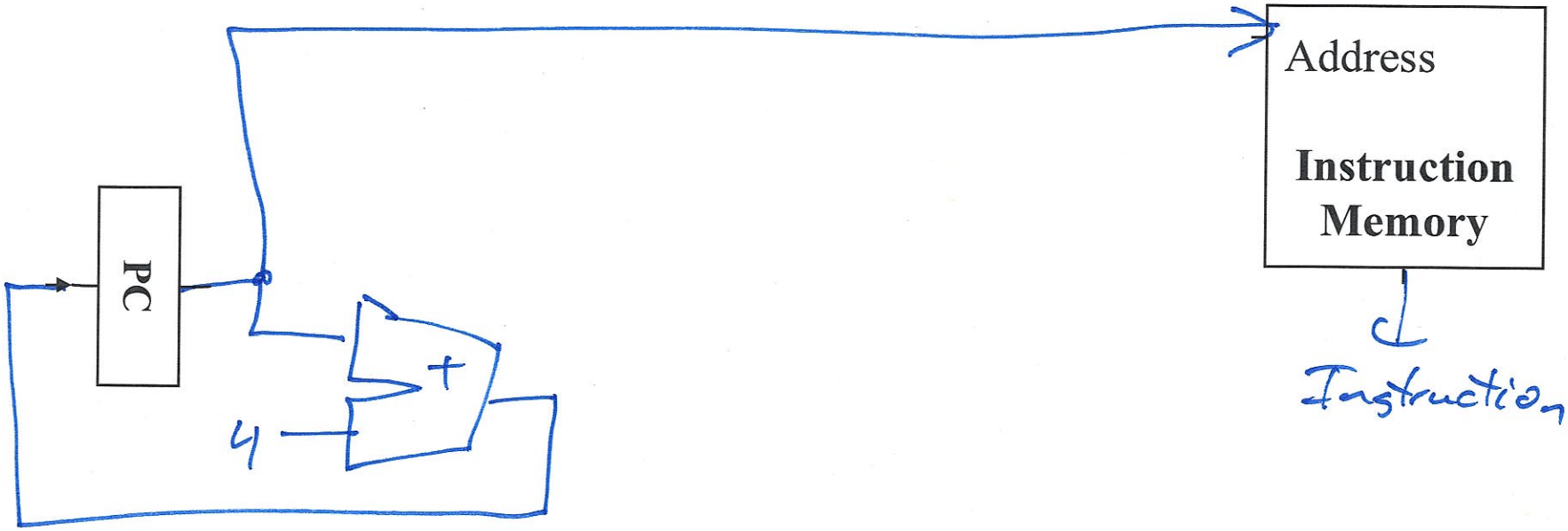
(hold): A = A; I++;

(init): A = Din; I++;



Instruction Fetch

Instruction = MEM[PC];
PC = PC+4;



Add/Subtract RTL

Add instruction: `ADD Rd, Rn, Rm`

Instruction = Mem[PC];
Reg[Rd] = Reg[Rn] + Reg[Rm];
PC = PC + 4

Subtract instruction: `SUB Rd, Rn, Rm`

Instr = Mem[PC]
Reg[Rd] = Reg[Rn] - Reg[Rm]
PC = PC + 4

